Project 0 — Bake-Off: extracting F1 driver facts

CS496 AI Engineering. Team: Mohamed Aziz Touhami, Tarak Ben Sedrine, Ilyes Fatnassi, Farah Marnissi

1. The task and one example item

Each item is a short biography of a real Formula 1 driver. The model must return **one line of JSON** with 6 facts: nationality, debut_year, career_points, era, win_percentage and podium_percentage. Two things make it harder than copying: (1) the **era** must be classified from debut_year using 5 fixed ranges (Golden 1950–69, Grand Prix 1970–89, Modern 1990–2006, Turbo-Hybrid 2007–21, Current 2022+), and (2) the two **percentages are never written in the text**: only wins, podiums and starts are, so the model must compute them. Each bio also contains one **distractor** sentence with another driver's real career points. Scoring is automatic: exact match for text and integers, a ±0.1 tolerance for percentages; an item is correct only if all 6 fields are correct.

Example (d01). "Lewis Hamilton is a British driver who made his Formula 1 debut in 2007 … Across 393 Grand Prix starts, Hamilton took 106 race wins and scored 5201.5 career championship points … he also picked up 206 podium finishes … He is often compared to Stirling Moss, whose career total sits at 186.64 championship points." → expected {"nationality":"British", "debut_year":2007, "career_points":5201.5, "era":"Turbo-Hybrid Era", "win_percentage":27.0, "podium_percentage":52.4}. The last sentence is the distractor.

2. The data

The 50 drivers come from f1_drivers_database.csv, a table compiled from public F1 statistics (Wikipedia driver pages and official records). Bios and answers are **generated mechanically** from the CSV by src/build_items.py (fixed seed), so every label follows directly from the source row. The frozen file data/items.jsonl was then **checked by script**: exact match against the CSV, word counts, the distractor never names the subject, and no distractor's points equal the subject's. **No dev/test split:** the prompt was written and frozen before the scored runs and was never tuned on the results, so all 50 items are one test set.

3. Setup

	Cheap API model	Open-weights model, run by us
Model	openai/gpt-oss-20b	phi4-mini (Microsoft Phi-4-mini)
Provider	Groq, OpenAI-compatible API	Ollama on our own laptop
Run date (UTC)	2026-09-28	2026-10-20
Settings	temperature 0, max 900 output tokens	temperature 0, max 900 output tokens
Hardware	(hosted)	Intel i7-10750H (6 cores), 11.8 GB RAM, no GPU, Windows 11

Both models got **the same prompt (src/prompt.txt), the same 50 items in the same order and the same strict parser** (json.loads, no repair, no retries; a parse error counts as wrong). The prompt lists the 6 fields, the era ranges and the formulas, and says: "Output ONLY a single line of valid JSON. No markdown code fences." The top-tier API model in the brief was dropped from our scope, so this is a two-model comparison. phi4-mini was chosen because qwen3:8b took over 180 s for a single item on this CPU.

4. Results

Model	Correct	Parse errors	p50 latency	p95 latency	Tokens in / out	Cost per 1k requests
gpt-oss-20b (Groq)	**49/50**	0	5,048 ms	8,792 ms	687 / 391	**$0.26**
phi4-mini (local)	**0/50**	50	13,224 ms	25,070 ms	619 / 51	**$0.61** + our time

Refusals and timeouts: 0 for both. phi4-mini ran at **7.8 tokens/s**. Results come from results/per_item.csv (one row per item per model) and results/summary.csv. Neither model was ever fooled by the distractor sentence.

Wrong answers, gpt-oss-20b (only 1 of 50 was wrong): d03 Max Verstappen, era: expected "Turbo-Hybrid Era", got "Modern Era". It read debut_year 2015 correctly but applied the range wrongly; 2015 is not a boundary year.

Wrong answers, phi4-mini (3 of 50 shown; all 50 fail the same way): d01 Lewis Hamilton, d02 Michael Schumacher and d03 Max Verstappen all return their JSON wrapped in a ```json … ``` code fence, which the prompt forbids, so the strict parser rejects them. Looking under the fence (diagnostic only, not the score): the JSON is valid, but **win_percentage and podium_percentage are null on all 50 items** (the model never does the calculation), era is right on 37/50 (d03 says "Current Era"), and career_points on 48/50. So even with the fences removed, phi4-mini would still score 0/50.

5. Our choice, and when we would change it

We would choose gpt-oss-20b on Groq. It is right on 49/50 items, about 2.6× faster at p50 and 2.3× cheaper per request than our local setup, and it follows the output format. phi4-mini on a CPU laptop fails the format on every item and never computes the percentages. **We would change our choice** if (1) the data could not leave our machines (privacy, offline use) — then we would self-host, but with a larger model on a GPU, not phi4-mini on a CPU; (2) a self-hosted setup reached more than about 570 requests per hour (under ~6.3 s per request) while staying accurate, which makes it cheaper than the API (see section 6); or (3) Groq's price, rate limits or availability got worse, or the model behind the name changed. We would re-run these 50 items before any switch.

6. Cost at 100× traffic and the break-even volume

API: tokens from the usage field × Groq list price ($0.10 in / $0.50 out per 1M tokens, checked 2026-10-20) = 687 × $0.10/1M + 391 × $0.50/1M = **$0.00026 per request**: **$0.26 per 1k today, $26.44 at 100× traffic (100k requests)**. **Self-hosted:** hardware cost per hour ÷ requests per hour. Assumptions: a $1,200 laptop over 3 years at 8 h/day ($0.137/h) + 70 W at $0.15/kWh ($0.011/h) = **$0.15/h**; at 14.6 s per request that is 246 requests/h, so **$0.61 per 1k** plus our time (about 5 minutes at $20/h per run). At 100× traffic: about $61 and 406 hours (17 days) of continuous running, versus $26 and no waiting for the API.

Break-even: self-hosted cost per request = $0.15/h ÷ requests per hour. It equals the API's $0.000264 only at 0.15 ÷ 0.000264 ≈ **567 requests per hour**. Our laptop does 246, so **on this hardware there is no break-even volume**: the API is cheaper at every volume, and our time only widens the gap. Self-hosting starts to pay off with hardware at least 2.3× faster for the same cost per hour (e.g. a GPU), and only if the model also becomes accurate.